Universidad Técnica Estatal de Quevedo
Facultad de Ciencias de la Computación – Carrera de Software

Aplicaciones Distribuidas (ISR-701) – Séptimo nivel, paralelo B

FORM-FB-01: Fichas bibliográficas

Fundamentos de sistemas distribuidos y arquitecturas distribuidas

Benites Pérez Dariem Alberto
Docente: Gleiston Guerrero Ulloa, PhD
8 de noviembre de 2026

1. Ficha 1

Referencia (IEEE): J. L. Bação and S. Guerreiro, "Consistency challenges in event-driven microservices: a literature review on data, process, and system evolution," *Computing*, vol. 108, no. 7, 2026, Art. no. 110. DOI: 10.1007/s00607-026-01701-5

DOI: 10.1007/s00607-026-01701-5

Tipo de fuente: Artículo de revista. *Computing*, Springer. ISSN 0010-485X, e-ISSN 1436-5057.

Resumen: El artículo revisa los desafíos de consistencia en microservicios basados en eventos. El problema es que el conocimiento está fragmentado y no hay consenso sobre cómo definir y mantener la consistencia de datos, procesos y sistemas en arquitecturas distribuidas. Los autores hicieron una revisión sistemática de literatura siguiendo las directrices de Kitchenham y analizaron 38 estudios de Web of Science para clasificar los enfoques existentes. Encontraron tres tipos de consistencia (datos, procesos y evolución del sistema), describieron las estrategias de cada uno y señalaron un vacío importante: la desconexión entre el diseño de los procesos de negocio y su ejecución.

Dato concreto: «Unlike ACID transactions, Sagas do not provide isolation and instead operate under eventual consistency» [1, Sec. 2.6, p. 6].

Nota personal: Este artículo me ayudó a conectar la teoría de sistemas distribuidos con la gestión de procesos de negocio y a entender por qué fallan los microservicios cuando no se alinean los datos, los procesos y la evolución de la arquitectura. Lo usaré en TA-IND-01, en la parte de CAP y PACELC, para explicar qué consistencia elige el sistema y qué pasa cuando falla un paso (tolerancia a fallos).

2. Ficha 2

Referencia (IEEE): A. Nicolas-Plata et al., "A service mesh approach to integrate processing patterns into microservices applications," *Cluster Computing*, vol. 27, no. 6, pp. 7417–7438, 2024. DOI: 10.1007/s10586-024-04342-5

DOI: 10.1007/s10586-024-04342-5

Tipo de fuente: Artículo de revista. *Cluster Computing*, Springer. ISSN 1386-7857, e-ISSN 1573-7543.

Resumen: Los gestores de malla de servicios actuales solo manejan el patrón de canalización (pipes & filters) y delegan los demás patrones al gestor de contenedores, lo que complica combinarlos. Para resolverlo, los autores proponen las Eblocks, unidades que integran de forma implícita los componentes de la malla (descubrimiento, autenticación y monitoreo) junto con la lógica de procesamiento. Con un caso de estudio real lograron desplegar patrones como manager-worker y patrones internos dentro de cada Eblock, con menores tiempos de servicio y mayor rendimiento que la misma solución montada con Istio sobre Kubernetes.

Dato concreto: «a service mesh (e.g., Istio) is focused on the communication and interactions between services within a microservices architecture» [2, Sec. 6, p. 7431].

Nota personal: Me llamó la atención que Istio depende de Kubernetes para crear réplicas, mientras que las Eblocks llevan las tareas de la malla dentro de cada bloque sin depender tanto del clúster. Me sirvió para conocer otra forma de optimizar flujos de procesamiento intensivos en microservicios. Lo usaré en TA-IND-01, en los modelos de comunicación y en la arquitectura del sistema, para explicar cómo se hablan los servicios entre sí.

3. Ficha 3

Referencia (IEEE): V. Velepucha and P. Flores, "A Survey on Microservices Architecture: Principles, Patterns and Migration Challenges," *IEEE Access*, vol. 11, pp. 88 339–88 358, 2023. DOI: 10.1109/ACCESS.2023.3305687

DOI: 10.1109/ACCESS.2023.3305687

Tipo de fuente: Artículo de revista. *IEEE Access*, IEEE. e-ISSN 2169-3536.

Resumen: El trabajo aborda la falta de conocimiento sobre los principios teóricos para descomponer sistemas monolíticos en microservicios. Los autores revisan la literatura y relacionan los principios de modularización y de ocultamiento de información de David Parnas (1972) con las recomendaciones de Martin Fowler y Sam Newman; también analizan patrones de diseño y casos de estudio. Proponen una comparación que unifica estos criterios, un resumen de las ventajas y desventajas de ambos enfoques y pautas para enfrentar los desafíos técnicos y organizacionales de la migración.

Dato concreto: En 2002, en Amazon, «all work teams had to expose their data and business functionality through services» [3, Sec. V-A, p. 88355].

Nota personal: Este análisis me permitió conectar las bases históricas de la ingeniería de software (como el ocultamiento de información) con las tendencias modernas de desarrollo. Me otorga una visión crítica para justificar cuándo vale la pena migrar a microservicios y cómo estructurar equipos autónomos y sistemas desacoplados de forma sostenible. Lo usaré en la introducción y en la arquitectura del sistema de TA-IND-01 para explicar qué es un monolito, qué son los microservicios y poner el caso de Amazon como ejemplo.

4. Ficha 4

Referencia (IEEE): K. Senjab et al., "A survey of Kubernetes scheduling algorithms," *Journal of Cloud Computing*, vol. 12, no. 1, 2023, Art. no. 87. DOI: 10.1186/s13677-023-00471-1

DOI: 10.1186/s13677-023-00471-1

Tipo de fuente: Artículo de revista. *Journal of Cloud Computing*, Springer. e-ISSN 2192-113X.

Resumen: El artículo trata la gestión de recursos en clústeres que orquestan contenedores con Kubernetes, donde el planificador decide en qué nodo se ejecuta cada contenedor. Los autores hicieron una revisión de literatura y clasificaron los algoritmos de planificación en cuatro categorías: planificación genérica, optimización multiobjetivo, enfoques basados en inteligencia artificial y esquemas con autoescalado. Para cada categoría identifican fortalezas, limitaciones y temas pendientes, y concluyen que aún hay margen de mejora, por lo que el trabajo futuro debe enfocarse en desarrollar nuevos algoritmos y mejorar los existentes.

Dato concreto: «We initially identified a total of 124 studies from the database searches» [4, p. 4]; «our review includes 47 studies on Kubernetes scheduling algorithms published in the last 5 years» [4, p. 5].

Nota personal: Gracias a este artículo tengo un panorama de cómo evolucionan los algoritmos de planificación de Kubernetes más allá de la asignación estática de recursos, sobre todo con inteligencia artificial y autoescalado. Lo usaré en TA-IND-01, en tolerancia a fallos y en las transparencias de migración y reubicación, para explicar cómo Kubernetes mueve y vuelve a levantar los contenedores.

5. Ficha 5

Referencia (IEEE): G. Blinowski et al., "Monolithic vs. Microservice Architecture: A Performance and Scalability Evaluation," *IEEE Access*, vol. 10, pp. 20 357–20 374, 2022. DOI: 10.1109/ACCESS.2022.3152803

DOI: 10.1109/ACCESS.2022.3152803

Tipo de fuente: Artículo de revista. *IEEE Access*, IEEE. e-ISSN 2169-3536.

Resumen: Este artículo investiga si la migración de sistemas monolíticos a microservicios realmente beneficia a pequeñas empresas o si es una tendencia idealizada. El estudio implementó una aplicación web de referencia en cuatro versiones combinando dos arquitecturas (monolito y microservicios) y dos tecnologías (.NET y Java), evaluándolas mediante experimentos controlados locales y en la nube de Azure. Los hallazgos principales demuestran que un monolito supera en rendimiento a los microservicios en una sola máquina, que el escalado vertical es más rentable que el horizontal, y que Java aprovecha mejor las máquinas potentes en servicios de cómputo intensivo, mientras que .NET rinde mejor con servicios livianos en hardware de poca capacidad.

Dato concreto: «on a single machine, a monolith performs better than its microservice-based counterpart»; «scaling out beyond a certain number of instances degrades the application performance» [5, Sec. VIII, p. 20372].

Nota personal: Analizar este estudio me ayudó a entender que no siempre debo adoptar microservicios por moda. Me dio una perspectiva técnica muy clara sobre cuándo vale la pena escalar verticalmente un monolito frente al costo y la sobrecarga de la infraestructura distribuida. Lo usaré en el análisis comparativo y en la postura crítica de TA-IND-01 para opinar si al sistema elegido le conviene su arquitectura o si un monolito le bastaría.

6. Declaración de uso de inteligencia artificial

Herramienta:	Claude (Anthropic), modelo Claude Opus 5.5, en la aplicación Claude Code.
Para qué se usó:	Para que me ayude corrigiendo y aclarando partes de los borradores me ayudó a analizar en qué sección de la próxima tarea podría usar cada fuente.
Cómo se verificó:	Abrí cada DOI manualmente y sus datos (autores, año, título, revista, volumen, número y páginas) se compararon con la página de la editorial. Cada cita textual se buscó en el PDF del artículo para confirmar la sección y la página. Leí y revisé el documento final antes de entregarlo.

Referencias

[1] J. L. Bação and S. Guerreiro, "Consistency challenges in event-driven microservices: a literature review on data, process, and system evolution," *Computing*, vol. 108, no. 7, 2026, Art. no. 110. DOI: 10.1007/s00607-026-01701-5

[2] A. Nicolas-Plata, J. L. Gonzalez-Compean, and V. J. Sosa-Sosa, "A service mesh approach to integrate processing patterns into microservices applications," *Cluster Computing*, vol. 27, no. 6, pp. 7417–7438, 2024. DOI: 10.1007/s10586-024-04342-5

[3] V. Velepucha and P. Flores, "A Survey on Microservices Architecture: Principles, Patterns and Migration Challenges," *IEEE Access*, vol. 11, pp. 88 339–88 358, 2023. DOI: 10.1109/ACCESS.2023.3305687

[4] K. Senjab, S. Abbas, N. Ahmed, and A. u. R. Khan, "A survey of Kubernetes scheduling algorithms," *Journal of Cloud Computing*, vol. 12, no. 1, 2023, Art. no. 87. DOI: 10.1186/s13677-023-00471-1

[5] G. Blinowski, A. Ojdowska, and A. Przybyłek, "Monolithic vs. Microservice Architecture: A Performance and Scalability Evaluation," *IEEE Access*, vol. 10, pp. 20 357–20 374, 2022. DOI: 10.1109/ACCESS.2022.3152803